SISTEMAS OPERACIONAIS

Persistência: arquivos, discos e sistemas de arquivos

Teoria e demonstrações práticas em C

Ciência da Computação, 2º período

Universidade Federal de Ciência e Inovação do Rio de Janeiro (UFCIRJ)

Prof. Frederico Luís Cabral

Roteiro das aulas

- **Aula 1:** persistência, E/S, HDDs/SSDs, arquivos, diretórios e interface POSIX.
- **Aula 2:** blocos, inodes, alocação, cache, `fsync()`, journaling e integridade.
- **Quatro programas C:** escrever/ler, `lseek`, metadados/links, `fsync`.
- **Princípio didático:** entender a operação, prever o resultado e conferir no terminal.

Ideia-chave: Não usamos slides das aulas anteriores: numeração e estrutura são novas.

AULA 1

Do processo ao arquivo

Interfaces, dispositivos e a ideia de persistência.

Pergunta inicial: o que sobrevive?

Para discutir com a turma

1. Um programa cria uma variável `int x = 42;` e termina. Onde está o valor?
2. O mesmo programa grava 42 em um arquivo e termina. O que muda?
3. Que componentes participam do caminho entre `write()` e o armazenamento?

Responda primeiro; confira no código ou no terminal em seguida.

Persistência: definição operacional

- Dados persistentes continuam disponíveis **após o encerramento do processo**.
- Memória RAM é volátil; SSD e HDD são dispositivos de armazenamento não volátil.
- O sistema operacional apresenta a abstração de **arquivos e diretórios**.
- Escrever um arquivo não implica, por si só, que os dados já estejam duravelmente registrados no dispositivo.

Ideia-chave: Persistência é diferente de duração da variável e de garantia imediata contra falhas de energia.

RAM e armazenamento

Memória principal (RAM)

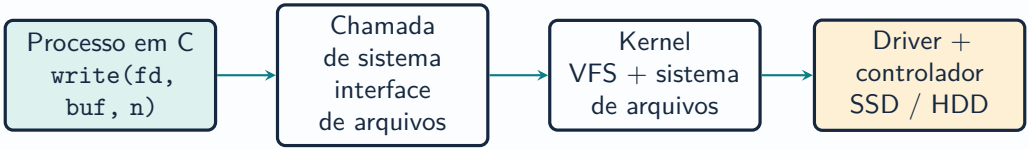
- Acesso rápido e diretamente endereçável pela CPU.
- Guarda código, pilha, heap e caches.
- Perde o conteúdo quando não há alimentação elétrica.

Armazenamento (SSD/HDD)

- Acesso por dispositivos e controladores.
- Guarda arquivos, diretórios e metadados.
- Não depende de o processo continuar executando.

Um arquivo pode ser lido por diferentes processos, desde que haja permissões adequadas.

O caminho de uma chamada de escrita



O código da aplicação não precisa conhecer os endereços físicos dos blocos do disco.

O que o sistema operacional abstrai?

- **Dispositivo:** esconde comandos específicos de cada controlador.
- **Arquivo:** oferece uma sequência de bytes e uma interface uniforme.
- **Diretório:** associa nomes a arquivos e outros diretórios.
- **Descritor:** número usado pelo processo para acessar um arquivo aberto.
- **Permissões:** controlam quais operações são autorizadas.

Arquitetura simplificada de E/S

- **Controlador:** recebe comandos do sistema operacional e opera o dispositivo.
- **Registradores:** expõem dados, estado e comandos (visão conceitual).
- **Driver:** traduz requisições genéricas em operações específicas.
- **Interrupção:** pode avisar à CPU que uma operação foi concluída.
- **DMA:** permite transferências entre dispositivo e memória com menor intervenção da CPU.

Polling e interrupções

Polling

- A CPU consulta repetidamente o estado do dispositivo.
- Pode ser simples e útil quando a espera é muito curta.
- Pode desperdiçar ciclos aguardando conclusão.

Interrupções

- O dispositivo sinaliza eventos ao processador.
- A CPU pode executar outras tarefas enquanto espera.
- Há custo de tratamento da interrupção.

Na prática, sistemas podem combinar técnicas conforme o dispositivo e a carga.

DMA: por que faz diferença?

- Sem DMA, a CPU pode precisar participar de transferências repetidas de dados.
- Com DMA, o controlador transfere blocos entre o dispositivo e a memória.
- A CPU continua responsável por configurar, coordenar e tratar a conclusão da operação.
- DMA não elimina drivers, interrupções, buffers ou outras responsabilidades do SO.

HDD e SSD: mecanismos físicos

HDD

- Pratos magnéticos e cabeçote mecânico.
- Latência de busca, rotação e transferência.
- Acesso aleatório geralmente mais caro.

SSD

- Memória flash, sem cabeçote mecânico.
- Acesso aleatório costuma ser muito mais rápido.
- Controlador realiza tradução e gerenciamento interno.

Os dois podem oferecer ao SO uma interface de blocos, apesar da implementação física distinta.

Custos de acesso a um HDD

- **Seek:** movimentação do cabeçote até a trilha.
- **Latência rotacional:** espera até que o setor passe sob o cabeçote.
- **Transferência:** leitura ou escrita dos bytes.
- A localização dos dados pode afetar muito o desempenho de operações pequenas.

Por que o SSD também precisa de gerenciamento?

- Flash organiza dados em páginas; a eliminação ocorre em unidades maiores (blocos de apagamento).
- O controlador mantém uma camada de tradução de endereços, a FTL.
- Existe *wear leveling*: distribuição das escritas para reduzir desgaste localizado.
- Desempenho de SSD não significa escrita instantânea nem ausência de falhas.

Teste rápido: E/S e armazenamento

Para discutir com a turma

1. Por que um HDD tende a penalizar mais acessos aleatórios do que um SSD?
2. Qual é a função de um driver de dispositivo?
3. Qual a diferença entre esperar por *polling* e ser avisado por interrupção?

Responda primeiro; confira no código ou no terminal em seguida.

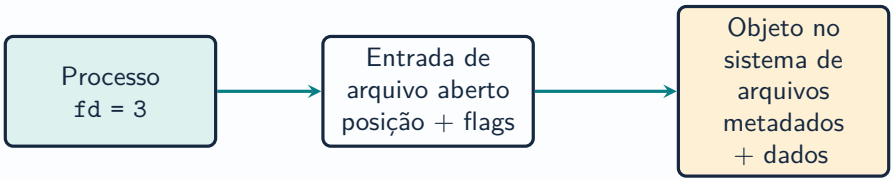
Arquivo: uma sequência de bytes

- Para a interface POSIX, um arquivo regular pode ser visto como uma sequência de bytes.
- O sistema de arquivos associa a essa sequência tamanho, permissões e outros metadados.
- O nome do arquivo pertence à organização de diretórios; não é o conteúdo do arquivo.
- Dois processos podem abrir o mesmo arquivo em momentos diferentes.

Descritores de arquivos

- `open()` retorna um inteiro: o **descritor de arquivo** do processo.
- Esse número referencia uma entrada mantida pelo sistema operacional.
- `read()`, `write()`, `lseek()` e `close()` operam sobre esse descritor.
- Por convenção, `0 = stdin`, `1 = stdout`, `2 = stderr`.

O descritor não é o nome do arquivo



O número do descritor é válido no contexto do processo. O nome foi usado para localizar o arquivo na abertura.

Principais chamadas POSIX

```c
int fd = open("registro.txt", O_RDONLY);
if (fd == -1) { perror("open"); return 1; }

char dados[128];
ssize_t n = read(fd, dados, sizeof(dados));
if (n < 0) perror("read");

close(fd);
```

`read()` devolve o número de bytes lidos; zero indica fim do arquivo.

Flags de abertura: decisões do processo

- `O_RDONLY`, `O_WRONLY` ou `O_RDWR`: modo de acesso.
- `O_CREAT`: cria se o arquivo não existir (com modo de permissões).
- `O_TRUNC`: zera o comprimento de um arquivo regular existente.
- `O_APPEND`: cada escrita é posicionada no final do arquivo.
- A função `close()` libera o descritor; não substitui uma exigência de durabilidade por `fsync()`.

Código 01: gravar e ler em processos diferentes

```
/* Trechos de 01_arquivo.c */
fd = open("registro.txt", O_WRONLY | O_CREAT |
         O_APPEND, 0644);
write(fd, mensagem, sizeof(mensagem) - 1);
close(fd);

/* Em outra execucao do programa: */
fd = open("registro.txt", O_RDONLY);
while ((n = read(fd, buffer, sizeof(buffer))) > 0)
    fwrite(buffer, 1, (size_t)n, stdout);
close(fd);
```

O arquivo de código contém verificações de erro; estes são somente os trechos fundamentais.

DEMONSTRACAO 01

Acompanhar no terminal

Diretório: `SO/05_persistencia/exemplos`

```
make
./01_arquivo gravar
./01_arquivo ler
./01_arquivo gravar
./01_arquivo ler
cat registro.txt
```

Observar: A segunda gravação acrescenta uma linha: o arquivo continua existindo após o primeiro processo terminar.

Para discutir com a turma

1. Por que executar `./01_arquivo ler` em outro processo não apaga a informação?
2. Por que a segunda execução de `gravar` não substitui a primeira linha?
3. O conteúdo exibido após `write()` demonstra, sozinho, resistência a uma falha elétrica?

Responda primeiro; confira no código ou no terminal em seguida.

Posição de leitura/escrita: deslocamento

- Um arquivo aberto possui uma posição corrente de leitura/escrita (*offset*).
- `read()` e `write()` normalmente avançam essa posição.
- `lseek(fd, deslocamento, origem)` altera a posição.
- `SEEK_SET` usa o início; `SEEK_CUR`, a posição atual; `SEEK_END`, o fim.
- `lseek()` muda a posição, **não lê nem escreve bytes**.

Código 02: substituindo um byte

```
/* O arquivo recebe inicialmente "ABCDEFGHIJ". */
write(fd, "ABCDEFGHIJ", 10);
lseek(fd, 4, SEEK_SET); /* indice 4 = quinta letra */
write(fd, "x", 1);
lseek(fd, 0, SEEK_SET);
read(fd, buffer, 10);
printf("%s\n", buffer);
```

Posições começam em zero. O `x` substitui `E`, não é inserido antes dele.

DEMONSTRACAO 02

Acompanhar no terminal

Diretório: `SO/05_persistencia/exemplos`

```
./02_lseek
cat posicoes.txt
wc -c posicoes.txt
```

Observar: Conteúdo esperado: `ABCDxFGHIJ`; o arquivo continua com dez bytes.

Diretórios: nomes e hierarquia

- Diretórios organizam nomes de arquivos e de outros diretórios.
- Caminho absoluto: começa na raiz (ex.: `/home/aluno/notas.txt`).
- Caminho relativo: interpretado a partir do diretório corrente.
- `.` indica o diretório atual; `..` indica o diretório pai.
- Diretórios têm representação e metadados próprios no sistema de arquivos.

Terminal: explorando diretórios e arquivos

```
mkdir -p pasta_teste
cp registro.txt pasta_teste/copia.txt
pwd
ls -l pasta_teste
cat pasta_teste/copia.txt
```

Executar no diretório `exemplos`, após a demonstração 01.

Permissões de arquivo e de diretório

Arquivo regular

- **r:** ler conteúdo.
- **w:** modificar conteúdo, se outros controles permitirem.
- **x:** executar, quando aplicável.

Diretório

- **r:** listar entradas.
- **w:** alterar entradas (com condições).
- **x:** atravessar/buscar nomes.

As permissões se aplicam a usuário, grupo e outros; confira com `ls -l`.

Fechamento da Aula 1

Para discutir com a turma

1. Explique o percurso simplificado `write()` → kernel → dispositivo.
2. Como `open()`, `read()`, `write()` e `close()` se relacionam?
3. Por que `lseek()` não é uma operação de inserção?
4. Qual a diferença prática entre HDD e SSD para acessos aleatórios?

Responda primeiro; confira no código ou no terminal em seguida.